

EXPERIMENT NO : 02

TITLE OF EXPERIMENT : Perform classification tasks over a given dataset and evaluate its performance

DATE OF PERFORMANCE :

EXPERIMENT No: 2

Title: Digital Classification using Multilayer Perceptron.

Aim: Perform classification tasks over a given dataset using direct functions and evaluate its performance. The MNIST handwritten digit dataset will be used for this purpose.

Theory: Classification is the process of recognizing, understanding and grouping objects or ideas into predefined categories (sub-populations). With pre-labeled training datasets, classification algorithms in machine learning can predict the category of unseen data. It is a supervised learning technique in which the algorithm learns from labeled input and output pairs to classify new observations into various classes. (For example: spam/not spam, yes/no, digit levels (0-9)).

Types of classification:

- **Binary Classification:** Two output classes (e.g., spam or not spam).
- **Multi-Class Classification:** More than two output classes(eg. Digit recognition (0-9)).

In this experiment, we will implement a multi-class classification task using the MNIST handwritten digit dataset. The dataset consists of 70,000 grayscale images of handwritten digits (0-9) each size of 28x28 pixels. It is a benchmark dataset widely used for evaluating image classification algorithms in computer vision and deep learning.

Multilayer Perceptron(MLP): The model used in this experiment is multilayer perceptron, a type of feedforward neural network made up of an input layer, one or more hidden layers and an output layer. Each 28x28 image is flattened into a vector of 784 input values. Hidden layers use a non-linear activation function such as ReLU to learn patterns in the pixel data. The output layer has 10 neurons (one per digit 0-9) with the softmax activation, which converts the outputs into class probabilities that sum to 1- the predicted digit is the class with highest probability.

Tools and Dataset

Python Libraries Used: tensorflow/keras,numpy,matplotlib,sklearn.metrics.

Dataset: MNIST handwritten digits, loaded directly via `keras.datasets.mnist.load_data()` — 60,000 training images and 10,000 test images, each 28×28 pixels, grayscale (0–255).

Preprocessing needed: normalise pixel values to the range 0–1 (divide by 255); flatten each 28×28 image to a 784-length vector (or use a Flatten layer); one-hot encode the digit labels with `to_categorical()` if using `categorical_crossentropy` loss.

Preprocessing needed: normalise pixel values to the range 0–1 (divide by 255); flatten each 28×28 image to a 784-length vector (or use a Flatten layer); one-hot encode the digit labels with `to_categorical()` if using `categorical_crossentropy` loss.

Steps in Modeling a Neural Network with TensorFlow/Keras

1. **Creating a model** – Stack neural network layers using the Sequential or Functional API.
2. **Compiling the model** – Define loss function, optimizer, and evaluation metrics.
3. **Fitting the model** – Train the model on the dataset to find patterns.

Improvement strategies:

- **Model creation** – Add layers, increase neurons, change activation functions.
- **Compilation** – Try different optimizers (e.g., Adam instead of SGD).
- **Training** – Increase epochs or adjust batch size.

□ Program Code

- Import tensorflow, numpy, matplotlib.pyplot, and metrics from sklearn.
- Load the MNIST dataset: `(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()`.
- Normalise pixel values by dividing `X_train` and `X_test` by 255.0.
- Flatten each 28×28 image (or add a Flatten layer as the first layer of the model).
- One-hot encode `y_train` and `y_test` using `keras.utils.to_categorical()` (skip this if using `sparse_categorical_crossentropy`).
- Build a Sequential model: Flatten input layer -> one or more Dense hidden layers with ReLU activation -> Dense output layer with 10 units and softmax activation.
- Compile the model with `optimizer='adam', loss='categorical_crossentropy'` (or `'sparse_categorical_crossentropy'`), and `metrics=['accuracy']`.
- Fit the model on the training data, e.g. `model.fit(X_train, y_train, epochs=10)`, and store the returned history object.
- Evaluate the trained model on the test set using `model.evaluate(X_test, y_test)`.
- Plot training vs. validation accuracy and loss curves from the history object.(Optional)
- Generate predictions on the test set and build a confusion matrix and classification report using `sklearn.metrics.confusion_matrix()` and `classification_report()`.

Evaluating Performance

Common metrics for classification:

- **Confusion Matrix** – Summarizes correct and incorrect predictions.
- **Accuracy**-(Correct Predictions)/(Total Predictions).
- **Precision**-Proportion of correctly predicted positives.
- **Recall**- Proportion of actual positives correctly predicted.
- **F1 Score**- Harmonic mean of Precision and Recall, balancing both metrics.

		Predicted condition	
		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection

Output of Program:

Decision Tree Results

```

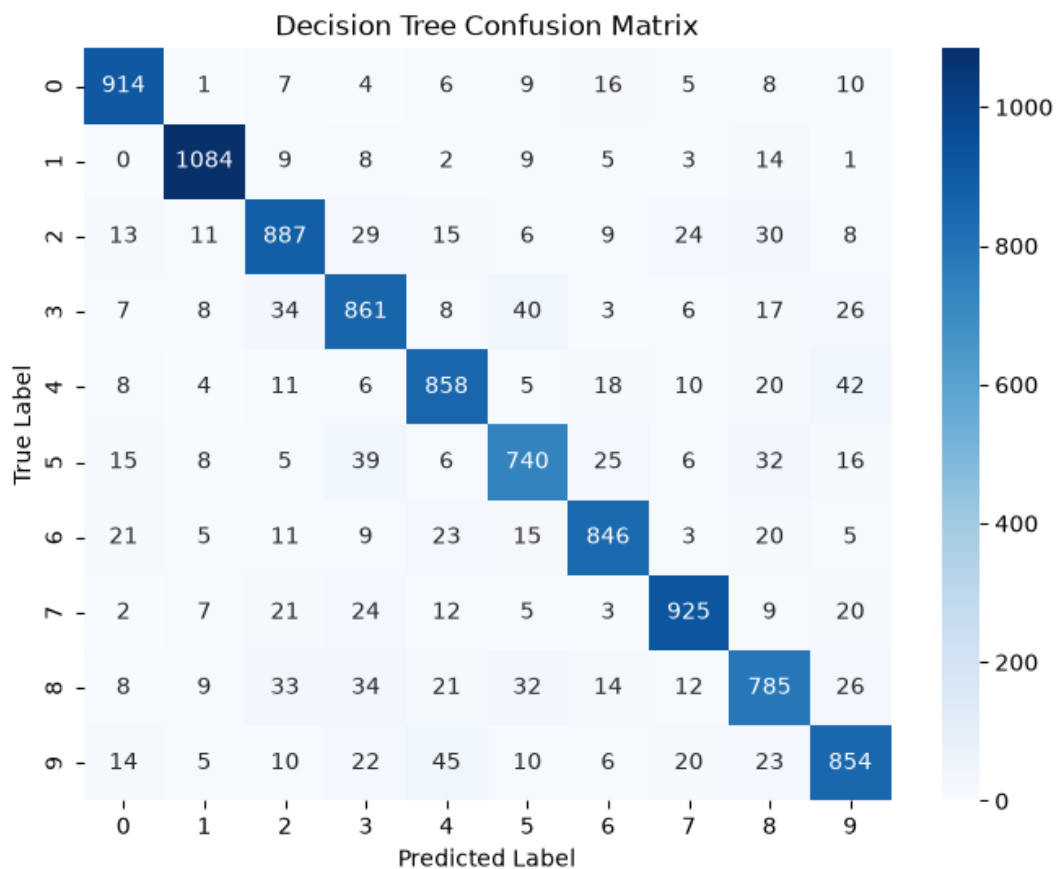
=====
Accuracy : 0.8754
Precision: 0.8753279394383177
Recall   : 0.8754
F1 Score : 0.8753090492142835

```

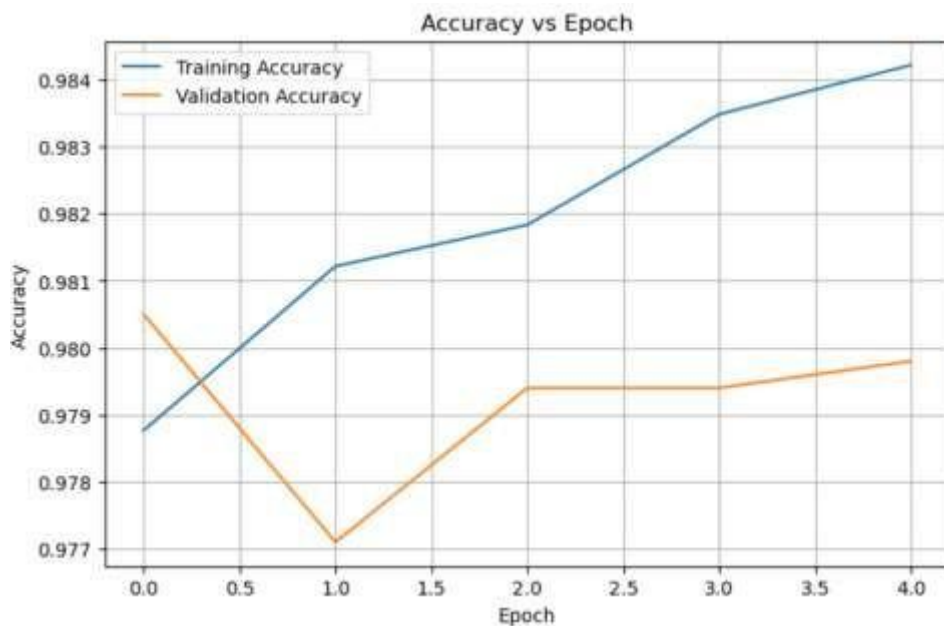
Classification Report

	precision	recall	f1-score	support
0	0.91	0.93	0.92	980
1	0.95	0.96	0.95	1135
2	0.86	0.86	0.86	1032
3	0.83	0.85	0.84	1010
4	0.86	0.87	0.87	982
5	0.85	0.83	0.84	892
6	0.90	0.88	0.89	958
7	0.91	0.90	0.91	1028
8	0.82	0.81	0.81	974
9	0.85	0.85	0.85	1009
accuracy			0.88	10000
macro avg	0.87	0.87	0.87	10000
weighted avg	0.88	0.88	0.88	10000

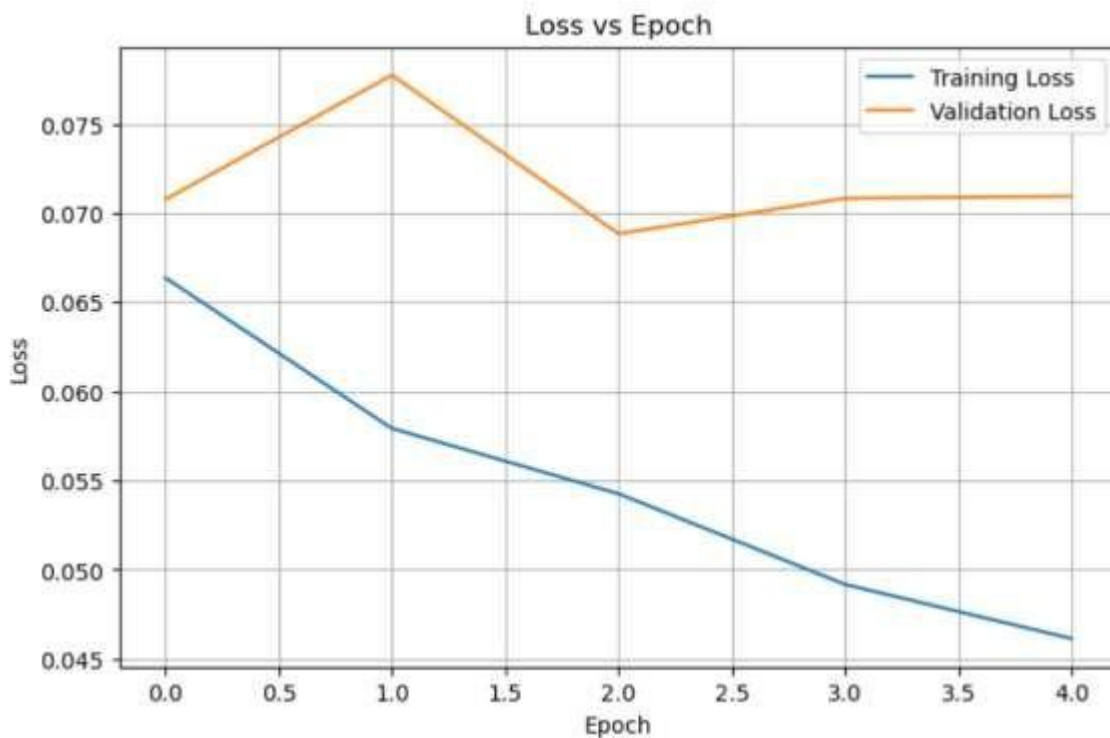
The Decision Tree model achieved an overall accuracy of 87.54% on the test dataset. The precision, recall, and F1-score are approximately 87.5%, indicating good classification performance across the 10 classes. The classification report shows that class-wise performance varies, with the highest F1-score of 0.95 for class 1 and the lowest of 0.81 for class 8.



The confusion matrix shows the comparison between the true labels and predicted labels for all 10 classes. The high values along the diagonal indicate that most samples are correctly classified, while the smaller off-diagonal values represent misclassified samples. This demonstrates that the Decision Tree model performs reasonably well across the different classes.



The graph shows the training and validation accuracy over 5 epochs. The training accuracy increases gradually from approximately 97.9% to 98.4%, while the validation accuracy remains around 97.7%–98.1%. This indicates that the model achieves high and relatively stable classification accuracy during training.



The graph shows the training and validation loss over 5 epochs. The training loss continuously decreases from approximately 0.066 to 0.046, indicating effective learning by the model. The validation loss varies slightly but remains relatively stable, indicating that the model maintains good performance on unseen data.

Conclusion: The model achieved a test accuracy of 87.54% on the MNIST dataset. From the confusion matrix, most samples were correctly classified, with the largest confusions occurring between digits 4 and 9, digits 3 and 5, and digits 5 and 3. The training accuracy increased gradually, while the validation accuracy remained relatively stable. The training loss continuously decreased, whereas the validation loss showed some fluctuation but remained stable overall. This indicates that the model learned effectively with no severe underfitting, although the difference between training and validation performance suggests slight overfitting. Performance could be improved by using more hidden layers or neurons, a better optimizer, regularization, and increasing the number of training epochs with appropriate early stopping.